

```
"""
```

```
BIST Al-Sat Sinyal Botu (Tüm Hisseler)
```

```
-----
```

```
Borsa İstanbul'da işlem gören TÜM hisseleri  
(XUTUM evreni: Yıldız, Ana ve  
Alt Pazar) RSI + Hareketli Ortalama  
stratejisiyle tarar, sinyal oluştuğunda  
Telegram'a bildirim gönderir.
```

```
Bu bot EMİR GÖNDERMEZ, sadece bildirim  
üretir. Alım-satım kararı ve  
işlemi tamamen kullanıcıya aittir. Yatırım  
tavsiyesi değildir.
```

```
Kullanılan kütüphane: borsapy  
(https://github.com/saidsurucu/borsapy)  
Not: borsapy kişisel/eğitim amaçlı kullanım  
için ücretsizdir.
```

```
"""
```

```
import os  
import json  
from datetime import datetime, timezone  
import requests  
import borsapy as bp
```

```
# -----
```

```
-----
```

```
# AYARLAR
```

```
# -----
```

```
-----

# Taranacak evren: XUTUM = BIST'te işlem
gören tüm hisseler
UNIVERSE = "XUTUM"

RSI_OVERSOLD = 40
RSI_OVERBOUGHT = 60
MA_PERIOD = 20

# AL: RSI düşük bölgede (gevşetilmiş eşik,
kesişim şartı yok – daha sık sinyal)
BUY_CONDITION = f"rsi < {RSI_OVERSOLD}"
# SAT: RSI yüksek bölgede (gevşetilmiş
eşik, kesişim şartı yok – daha sık sinyal)
SELL_CONDITION = f"rsi > {RSI_OVERBOUGHT}"

# Çok düşük hacimli/az işlem gören
hisseleri elemek için asgari hacim (TL)
MIN_VOLUME = 500_000

TELEGRAM_BOT_TOKEN =
os.environ.get("TELEGRAM_BOT_TOKEN")
TELEGRAM_CHAT_ID =
os.environ.get("TELEGRAM_CHAT_ID")

STATE_FILE = "last_signals.json"

# -----
# -----
# YARDIMCI FONKSİYONLAR
# -----
# -----
```

```
def load_state() -> dict:
    if os.path.exists(STATE_FILE):
        with open(STATE_FILE, "r") as f:
            return json.load(f)
    return {}

def save_state(state: dict) -> None:
    with open(STATE_FILE, "w") as f:
        json.dump(state, f)

def send_telegram_message(text: str) ->
None:
    if not TELEGRAM_BOT_TOKEN or not
TELEGRAM_CHAT_ID:
        print("UYARI: Telegram bilgileri
eksik, mesaj gönderilemedi.")
        print(text)
        return
    url =
f"https://api.telegram.org/bot{TELEGRAM_BOT
_TOKEN}/sendMessage"
    try:
        resp = requests.post(url, data=
{"chat_id": TELEGRAM_CHAT_ID, "text":
text}, timeout=20)
        if resp.status_code != 200:
            print(f"Telegram gönderim
hatası: {resp.text}")
    except Exception as e:
        print(f"Telegram gönderim
istisnası: {e}")
```

```

def run_scan(condition: str):
    """Verilen koşula uyan hisseleri XUTUM
    evreninde tarar."""
    try:
        df = bp.scan(UNIVERSE, condition,
            interval="15m")
    except Exception as e:
        print(f"Tarama hatası
        ({condition}): {e}")
        return []

    if df is None or len(df) == 0:
        return []

    results = []
    for _, row in df.iterrows():
        volume = row.get("volume", 0) or 0
        if volume < MIN_VOLUME:
            continue
        results.append({
            "symbol": row.get("symbol"),
            "price": row.get("price"),
            "rsi": row.get("rsi"),
        })
    return results

# -----
# -----
# ANA AKIŞ
# -----
# -----

```

```

def main():
    state = load_state()
    new_state = dict(state)
    messages = []

    buy_hits = run_scan(BUY_CONDITION)
    sell_hits = run_scan(SELL_CONDITION)

    seen_symbols = set()

    for hit in buy_hits:
        symbol = hit["symbol"]
        seen_symbols.add(symbol)
        if state.get(symbol) != "AL":
            messages.append(
                f"🟢 AL sinyali:
{symbol}\n"
                f"Fiyat: {hit['price']} TL
| RSI: {round(hit['rsi'], 1) if hit['rsi']
else '-'}"
            )
            new_state[symbol] = "AL"

    for hit in sell_hits:
        symbol = hit["symbol"]
        seen_symbols.add(symbol)
        if state.get(symbol) != "SAT":
            messages.append(
                f"🔴 SAT sinyali:
{symbol}\n"
                f"Fiyat: {hit['price']} TL
| RSI: {round(hit['rsi'], 1) if hit['rsi']
else '-'}"
            )
            new_state[symbol] = "SAT"

```

```

        )
        new_state[symbol] = "SAT"

        # Daha önce sinyal verilmiş ama artık
        koşulu sağlamayan hisseleri sıfırla
        # ki koşul tekrar oluştuğunda yeniden
        bildirim gitsin
        for symbol in list(new_state.keys()):
            if symbol.startswith("_"):
                continue # _heartbeat_date
            gibi özel anahtarları koru
            if symbol not in seen_symbols:
                new_state[symbol] = None

        if messages:
            full_message = f"📊 BIST Sinyal
            Botu ({len(messages)} yeni sinyal)\n\n" +
            "\n\n".join(messages)
            send_telegram_message(full_message)
            print(full_message)
        else:
            print("Bu taramada yeni sinyal
            yok.")

        # Günde bir kez, sinyal olsun olmasın,
        "bot çalışıyor" özet mesajı gönder
        # (state dosyasında "_heartbeat_date"
        ile takip edilir, spam yapmaz)
        today =
        datetime.now(timezone.utc).date().isoformat
        ()
        if new_state.get("_heartbeat_date") !=
        today:
            heartbeat = (

```

```
f"✅ Bot bugün ilk kez çalıştı
ve piyasayı taradı.\n"
f"Şu anda {len(buy_hits)}
hissede AL, {len(sell_hits)} hissede SAT "
f"koşulu görülüyor (hepsi daha
önce bildirilmiş olabilir).\n"
f"Bot düzenli çalışıyor, her
şey yolunda 👍"
)
send_telegram_message(heartbeat)
print(heartbeat)
new_state["_heartbeat_date"] =
today

save_state(new_state)

if __name__ == "__main__":
    main()
```